

Problem Statement 8: ShopPractice – Selenium-Java Test Automation Framework for an E-Commerce Practice Web Application

1. Hackathon Objective

This hackathon evaluates students on:

- *Selenium WebDriver fundamentals and browser automation*
 - *Page Object Model (POM) design pattern*
 - *TestNG framework – annotations, listeners, and DataProviders*
 - *Explicit wait strategy for handling dynamic elements*
 - *Screenshot capture on test failure*
 - *Clean, reusable, and maintainable framework design*
-

2. Problem Statement

Design and develop a Selenium-Java Automation Test Framework for a dedicated e-commerce automation practice web application.

The framework should automate end-to-end test scenarios across the following functional areas:

- *User login and logout*
- *Product listing and detail verification*
- *Shopping cart – add and manage items*
- *Order placement and order history verification*
- *Form validations – empty fields and invalid inputs*

GUVI

The framework must follow the Page Object Model pattern and must be structured as a reusable framework — not just a collection of standalone test scripts.

3. Application Under Test (AUT)

URL: <https://rahulshettyacademy.com/client>

No backend setup required. Students register a new account and directly automate this live practice application.

Key modules available for testing:

- **Auth** – Register, Login, Logout
 - **Products** – Product Listing, Add to Cart
 - **Cart** – View Cart, Remove Item, Checkout
 - **Orders** – View Order History, Order Detail
 - **My Account** – Profile Information
-

4. Test Modules and Scope

Teams must cover at least 3 out of the 5 modules listed below. Modules 1, 2, and 3 are mandatory. Modules 4 and 5 are optional.

4.1 Test Module 1 – User Authentication (Mandatory)

- *Verify successful login with valid registered credentials*
- *Verify login failure with wrong password and validate the error message*
- *Verify logout redirects to the login page*
- *Verify login with empty username and password shows validation messages*

4.2 Test Module 2 – Product Listing (Mandatory)

- *Verify the product dashboard loads with at least one product after login*

GUVI

- *Verify each product card shows a name and price*
- *Add a product to the cart from the listing page and verify the cart badge count updates*
- *Verify that multiple products can be added from the listing without navigating away*

4.3 Test Module 3 – Shopping Cart (Mandatory)

- *Navigate to the cart and verify all added products appear with correct names and prices*
- *Delete a product from the cart and verify it no longer appears*
- *Verify the total price in the cart reflects the correct sum of all items*
- *Proceed to checkout and verify the order is placed successfully*

4.4 Test Module 4 – Order History (Optional)

- *Navigate to the My Orders section and verify the most recent order appears*
- *Open an order detail page and verify the product name matches what was ordered*
- *Verify the order history page shows a date and order ID for each entry*

4.5 Test Module 5 – Form Validations (Optional)

- *Submit the login form with empty fields and verify the error messages shown*
- *Attempt registration with an already-registered email and verify the duplicate error*
- *Enter a password below the minimum required length and verify the validation message*

5. Must-Have Requirements

5.1 Page Object Model (POM)

- *Every page must have a dedicated Page class (LoginPage, DashboardPage, CartPage, OrderPage)*
- *Locators and action methods must be inside Page classes only*
- *Test classes must only call Page class methods — no findElement or click in test classes*
- *A BasePage class must have at least one shared utility (example: waitForElement)*

5.2 TestNG Configuration

- *testng.xml must define the test suite and list all test classes*
- *Use @BeforeMethod and @AfterMethod for browser launch and teardown*
- *Use at least one @DataProvider for login tests (valid and invalid credentials)*

5.3 WebDriver Management

- *Use WebDriverManager — no manual driver setup*
- *Browser name must be read from config.properties (chrome or firefox)*

5.4 Wait Strategy

- *Thread.sleep() is strictly not allowed — deduction will be applied*
- *Use WebDriverWait with ExpectedConditions for elements that load dynamically*

5.5 Configuration Management

- *config.properties must hold: browser, base URL, and timeout value*
- *A ConfigReader class must read and provide these values*
- *No hardcoded URLs or values in any test or page class*

5.6 Screenshot on Failure

- *Implement ITestListener to capture a screenshot automatically when a test fails*
- *Save screenshots with a timestamp in the filename inside a /screenshots/ folder*

5.7 Test Reporting

- *ExtentReports must generate an HTML report after every run*
- *Report must show test name, pass/fail status, and screenshot on failure*

Required Maven dependencies: selenium-java, testng, webdrivermanager, extentreports

6. Business Rules (Strictly Enforced)

- *Thread.sleep() anywhere results in a score deduction*

GUVI

- *Hardcoded URLs or credentials in any class other than config.properties results in a deduction*
 - *WebDriver code directly inside test methods results in a POM score deduction*
 - *Suite must run from testng.xml using: mvn test*
-

7. Good to Have

- *@DataProvider reading from a JSON or Excel file instead of inline data*
 - *Parallel execution in testng.xml (thread-count="2")*
 - *FluentWait for slow-loading product dashboard*
 - *Headless browser mode from config.properties*
 - *Retry on failure using IRetryAnalyzer*
-

8. Non-Functional Expectations

- *Clean separation between Page classes, Test classes, and config*
 - *No duplicate code — shared actions must go into BasePage or utilities*
 - *Meaningful method and variable names throughout*
 - *No Thread.sleep() — all timing handled via WebDriverWait*
 - *Code readability over quantity — a clean small framework beats a messy large one*
-

9. Hackathon Guidelines

- *Spend the first 20–30 minutes planning before writing any code*
- *Internet and AI usage is allowed — but every member must explain their own code*
- *Every member must contribute at least one Page class and one Test class*
- *No hardcoded URLs or credentials anywhere except config.properties*
- *Final walkthrough: 5 minutes per team, each member speaks about their own work*

10. Evaluation Criteria

- *Framework Design*
 - *Test Coverage*
 - *Selenium Proficiency*
 - *Data-Driven and Config Design*
 - *Reporting and Evidence*
 - *Team Collaboration*
-